Compatibilidad Android de Mascotas Municipales

Informe de avance del taller

Integrantes: Juan David, Juan Pablo, Daniel Segura y Ricardo.

Asignatura: Desarrollo Móvil · Fecha: 20 de octubre de 2026

1 Lo que llevamos hasta ahora

En este taller estamos trabajando con nuestra aplicación Mascotas Municipales. El objetivo es conservar sus funciones y comprobar que el mismo APK funcione en Android 7.0 y Android 15. No estamos creando una aplicación nueva ni cambiando versiones solo para que la compilación pase.

Hasta este momento revisamos la configuración del proyecto, corregimos la información de los aportes individuales y aplicamos una mejora en la carga de las fotos. El APK inicial y el modificado se instalaron y abrieron en los dos emuladores. La prueba de miniaturas pasó en ambos. Todavía falta completar los recorridos de las tres funciones elegidas, por eso presentamos este documento como avance.

Distribución del trabajo

Integrante	Responsabilidad acordada
Juan David	Cambios de código, rama de trabajo, registro técnico y comprobación de la mejora.
Daniel Segura	Auditoría de SDK, herramientas y dependencias; consolidación de la matriz técnica.
Juan Pablo	Investigación de riesgos, comparación de alternativas y registro del uso de IA.
Ricardo	Pruebas completas de las funciones en ambos Android y organización de sus evidencias.

Las tres funciones que vamos a conservar

- Inicio de sesión: entrar con una cuenta ciudadana ficticia y comprobar que se prepara el perfil.
- Registro y consulta de mascotas: guardar una ficha, confirmar la respuesta del servidor y volver a consultarla.
- Creación y consulta de reportes: enviar un reporte ficticio, confirmar su guardado y encontrarlo en la cuenta correspondiente.

El mapa, QR, accesibilidad y los flujos profesionales se conservan. No los eliminamos por no estar dentro de estos tres recorridos principales. Firebase sigue sin cambios de reglas ni servicios nuevos; no se habilitó facturación.

2 Auditoría y matriz técnica

La revisión se hizo sobre los archivos reales del proyecto, el APK generado y los manifiestos de las bibliotecas que resolvió Gradle. Estas son las versiones encontradas, no versiones propuestas para actualizar.

Elemento	Valor comprobado	Decisión y evidencia
minSdk	24 · Android 7.0	Se conserva. app/build.gradle.kts y APK.
targetSdk	33 · Android 13	Se conserva para este cambio. No convierte Android 15 en Android 13.
compileSdk	35 · Android 15	Se conserva como SDK para compilar. No aumenta por sí solo el mínimo instalable.
AGP / Gradle	9.3.1 / 9.5.0	Configuración raíz y wrapper. La compilación actual pasó.
JDK del lanzador	21.0.6	Salida de gradlew --version del entorno usado.
JDK del daemon	25	toolchainVersion=25 y registro del daemon con javaVersion=25.
Nivel Java del código	11	compileOptions. No es el JDK que ejecuta Gradle ni garantiza todas las API en Android 7.
Google Services	4.5.0	Plugin que procesa la configuración de cliente de Firebase.
Versión de la app	1.5 · versionCode 6	Se mantiene en el APK de esta comparación.

Dependencias y mínimos encontrados

Biblioteca o componente	Versión	API mínima del manifest local
Firebase Authentication	24.2.0	23
Cloud Firestore	26.6.0	23
Google Code Scanner	16.1.0	21 declarada; la guía actual pide 23 o superior.
osmdroid	6.1.20	1 declarada; no la tomamos como garantía de soporte real de Android 1.
AndroidX Core KTX / AppCompat	1.15.0 / 1.7.1	21 / 21
Lifecycle ViewModel / LiveData Core	2.6.2 / 2.6.2	14 / 14

Firebase BoM 34.19.0 coordina versiones; no es una biblioteca ejecutable con minSdk propio. La fusión del manifest pasó y el APK declara API 24. Eso permite instalarlo, pero no demuestra todas las funciones de cada biblioteca. Authentication y Firestore no exigen obligatoriamente Google Play Services en estas versiones; el lector QR sí tiene esa dependencia de ejecución. Fuentes [1], [4] y [5].

3 Riesgos y alternativas investigadas

Diferencia real de interfaz entre versiones

En las capturas vimos que Android 7 muestra los botones del sistema claros sobre un fondo también claro. El código fija ese fondo y solicita iconos oscuros con isAppearanceLightNavigationBars. Según Android, esa solicitud no tiene efecto antes de API 26. Por eso registramos un problema de contraste para revisar. La aplicación abre; no presentamos este hallazgo como un cierre de la app. Fuente [2].

Criterio	A Fondo según versión	B Fondo oscuro en ambas versiones
Propuesta	Antes de API 26 usar fondo oscuro; desde API 26 conservar fondo claro con iconos oscuros.	Usar barra oscura e iconos claros en los dos Android.
Ventaja	Conserva el aspecto actual donde los iconos oscuros están disponibles.	Reduce las diferencias de configuración entre versiones.
Riesgo y mantenimiento	Hay que probar las dos ramas y los modos de accesibilidad.	Cambia también el aspecto del Android moderno y requiere revisión visual.
Estado	Propuesta preferida para discutir y probar. No está implementada.	Alternativa comparada. No está implementada.

Las dos alternativas mantienen la misma aplicación y sus funciones. Ninguna exige retroceder bibliotecas. La preferencia por A todavía debe revisarse con el equipo; la mejora ya aplicada en este avance es la de memoria descrita en la siguiente sección.

Otros riesgos que sí corresponden al proyecto

- QR: depende de Google Play Services y de su módulo. Ya existe la consulta manual como respaldo; falta probar ambos caminos en los dos Android. Fuente [4].
- Mapa: osmdroid está archivado y las teselas necesitan red. Un fallo de certificados en Android 7 es una hipótesis que debe comprobarse con Logcat; no lo hemos observado. Fuente [7].
- Cambio de targetSdk: subirlo a 35 activa nuevas exigencias de interfaz en Android 15, entre ellas el comportamiento de borde a borde. No lo cambiamos para resolver un riesgo que no requería esa actualización. Fuente [6].
- Firebase remoto: README señala despliegues de reglas e índices pendientes. No consultamos la consola remota en esta revisión ni asumimos que esté actualizada.

Corrección del ejemplo sobre fechas

El ejemplo de java.time del aporte inicial no describe nuestra app. El código usa Calendar y DateFormat; no encontramos LocalDate ni LocalDateTime. Tampoco activamos coreLibraryDesugaring ni tenemos un Logcat con ese NoClassDefFoundError. Por eso quitamos esa afirmación del informe.

4 Cambio aplicado y resultados

El cambio se preparó en la rama codex/android-compatibilidad, creada desde el commit 4764e19. Antes de comenzar no había modificaciones en archivos versionados. Se conservaron los documentos y carpetas locales del equipo, y se guardó un APK inicial para comparar.

Qué se cambió y por qué

Las cuatro fotos de demostración tienen 1024 × 1024 píxeles. Antes se decodificaban siempre a 512 × 512, incluso en tarjetas de 76 dp. Ajustamos setPhoto en MainActivity para que las miniaturas carguen una resolución acorde con la tarjeta. La caché identifica tanto la foto como su muestreo, de manera que una miniatura no sustituya la imagen del detalle. El detalle conserva la resolución anterior. Fuente [3].

Comparamos ajustar el muestreo existente con agregar una biblioteca de imágenes. Aplicamos el primer camino porque es un cambio pequeño para cuatro recursos locales y no agrega dependencias. Agregar otra biblioteca podría ayudar con carga asíncrona, pero ampliaría el mantenimiento y las pruebas necesarias.

Prueba	Android 7.0 · API 24	Android 15 · API 35
Instalación y arranque del APK inicial	Correctos	Correctos
Instalación y arranque del APK modificado	Correctos	Correctos
Cuatro fotos con muestreo anterior	4.194.304 bytes · 4 MiB	4.194.304 bytes · 4 MiB
Cuatro miniaturas con el cambio	1.048.576 bytes · 1 MiB	1.048.576 bytes · 1 MiB
Reducción en esos bitmaps	75 %	75 %
Caché y resolución del detalle	Prueba aprobada	Prueba aprobada
Inicio de sesión completo	Pendiente	Pendiente
Registro y consulta de mascota	Pendiente	Pendiente
Creación y consulta de reporte	Pendiente	Pendiente

La instrumentación ejecutó el decodificador real de cada Android. En ambos emuladores, 76 dp equivalen a 152 píxeles. La comparación reproduce el muestreo anterior y comprueba la caché y las dimensiones del detalle. No mide un recorrido completo ni demuestra que toda la app use 75 % menos memoria.

Compilación e incidencias

Pasaron assembleDebug, lintDebug y assembleDebugAndroidTest. El último informe de lint registra 0 errores y 40 advertencias; no lo presentamos como una aplicación libre de advertencias. No apareció un error de compilación de la app. Hubo problemas de descarga, registro del SDK y ruta de los AVD; se resolvieron al preparar el entorno y no se atribuyen a incompatibilidades del código.

5 Evidencia en los dos Android

Estas capturas corresponden al mismo APK modificado. Muestran que la aplicación abrió el formulario de acceso. No prueban que ya se haya iniciado sesión ni que se hayan guardado mascotas o reportes.

Android 7.0 · API 24	Android 15 · API 35
	

Entorno: emuladores x86_64 de 720 × 1280 píxeles, densidad 320 y dos núcleos. Android 7 se configuró con 1536 MiB de RAM y Android 15 con 2048 MiB. No son mediciones en teléfonos físicos de gama baja.

Archivo utilizado: apk/MascotasMunicipales-compatibilidad.apk

SHA-256 del APK modificado:

`5C7462623551FD6C960C05C0FC4BB3DB8912CA7AB5E21EE18A60537712BD19C8`

Los archivos entorno.json de API 24 y API 35 registran exactamente esta misma huella.

Evidencia del proyecto: tests/evidencias-compatibilidad/api24 y api35 contienen instalacion.txt, inicio.txt, inicio.png, memoria-inicio.txt, fotos.txt y entorno.json. Las carpetas api24-inicial y api35-inicial conservan la comparación de instalación y arranque previa al cambio.

6 Respuestas técnicas de la primera parte

Pregunta 1 Mínimos de la app y sus dependencias

La app declara minSdk 24 y el APK confirma ese valor. En los manifiestos locales de las bibliotecas encontramos API 23 para Authentication y Firestore; API 21 para Code Scanner, AppCompat y Core KTX; y API 14 para las dos bibliotecas Lifecycle. En el AAR de osmdroid aparece 1, pero ese dato aislado no permite asegurar funcionamiento real en versiones tan antiguas. La guía de Code Scanner, además, pide API 23 o superior. Dejamos esa diferencia registrada en la matriz.

Lo comprobamos con app/build.gradle.kts, el informe de fusión del manifest, los manifiestos de los AAR resueltos y la inspección del APK. La fusión y la compilación pasaron, y el APK se instaló en API 24. Todavía falta probar todos los caminos de las bibliotecas dentro de los recorridos elegidos; no basta con mirar el mínimo declarado. Fuentes [1], [4] y [5].

Pregunta 2 Versiones que conservamos y motivo

Conservamos compileSdk 35 porque permite compilar contra esas API, sin obligar a instalarlas en el Android antiguo. Conservamos targetSdk 33 para no mezclar la mejora de memoria con cambios de comportamiento de la plataforma. No significa que Android 15 se comporte por completo como Android 13. Una actualización del objetivo a 35 necesitaría revisar también los márgenes y las barras del sistema. Fuentes [1] y [6].

También conservamos AGP 9.3.1, Gradle 9.5.0 y las bibliotecas actuales porque la compilación pasó y no encontramos una razón comprobada para retrocederlas. Diferenciamos el JDK del lanzador, 21.0.6, del daemon que ejecutó la compilación, 25. El nivel Java 11 del código es otra configuración distinta; por sí solo no garantiza que cualquier API de Java esté disponible en Android 7.

Pregunta 3 Diferencias de API e implementación compatible

El ejemplo que sí encontramos está en la barra de navegación del sistema: isAppearanceLightNavigationBars permite pedir iconos oscuros, pero no funciona antes de API 26. En Android 7 observamos iconos claros sobre el fondo claro de la app. Para mantener una sola aplicación proponemos condicionar el fondo por versión: oscuro en API 24 y conservar el comportamiento actual donde los iconos oscuros sí están disponibles. Comparamos esta idea con usar fondo oscuro en ambas versiones. La corrección todavía está pendiente; no la presentamos como implementada. Fuente [2].

La app ya separa recursos de tema en values y values-v31, que Android selecciona según la versión. También vimos una diferencia en el dibujo de la huella del encabezado, relacionada con la fuente de emoji del sistema. Esa variación visual no es evidencia de una función eliminada o de un cierre.

7 Respuestas técnicas y reproducción

Pregunta 4 Recursos y optimización concreta

Todavía no podemos afirmar qué operación consume más recursos en toda la aplicación. Falta comparar el mapa y los directorios durante su uso real. Lo que sí identificamos fue un gasto evitable en la carga de las fotos de las tarjetas. En la prueba aislada de ambos Android, los cuatro bitmaps pasaron de 4 MiB a 1 MiB, sin reducir las dimensiones del detalle. Esa es la optimización comprobada. No medimos todavía CPU, batería ni fluidez completa. Fuente [3].

Pregunta 5 Riesgos de retroceder bibliotecas

Retroceder una biblioteca puede dejar fuera correcciones de seguridad, cambios de mantenimiento o funciones que usa el proyecto; también puede crear conflictos con otras dependencias. El efecto depende de la versión concreta, así que no afirmamos que cualquier versión anterior sea insegura. En este avance no retrocedimos Firebase, osmdroid ni Code Scanner.

Para las imágenes descartamos añadir una biblioteca nueva y preferimos ajustar el muestreo existente. Para la interfaz comparamos un fondo condicionado por API con un fondo oscuro común. Tampoco elegimos bajar targetSdk como remedio general, porque eso no agrega una API que falte en Android 7. Subir minSdk dejaría fuera el dispositivo antiguo requerido. osmdroid requiere seguimiento aparte por estar archivado; eso no demuestra una vulnerabilidad concreta en la app. Fuentes [1] y [7].

Pregunta 6 Evidencia del mismo APK y las tres funciones

Los registros de API 24 y API 35 contienen la misma huella SHA-256 del APK modificado. También incluyen instalación, arranque, capturas y la prueba de fotos. Eso demuestra que se usó el mismo archivo para esas comprobaciones. No demuestra todavía que se completaron las tres funciones: faltan los recorridos de inicio de sesión, mascota y reporte en ambos Android. Ricardo debe añadir las evidencias de cada uno antes de dar esta respuesta por terminada.

Cómo repetir lo comprobado

Desde la carpeta del proyecto, con el SDK y los dos emuladores preparados:

```
gradlew.bat :app:assembleDebug :app:lintDebug :app:assembleDebugAndroidTest --offline
```

```
.\scripts\verify-android-compatibility.ps1 -Serial emulator-5554 -ExpectedApi 24
```

```
.\scripts\verify-android-compatibility.ps1 -Serial emulator-5556 -ExpectedApi 35
```

Los seriales son ejemplos: hay que usar los que muestre adb devices -l. No se debe recompilar entre las dos pruebas del mismo APK. La instrumentación se instala aparte y no forma parte del APK normal.

8 Uso de IA y trabajo pendiente

Usamos Codex como apoyo para revisar el código, contrastar la documentación, preparar la mejora, ejecutar comprobaciones y consolidar este informe. La redacción reúne el avance del grupo; no reemplaza la revisión de cada integrante ni su explicación en la sustentación.

Consulta o apoyo	Qué se aceptó o corrigió
Compatibilidad de fechas	Se descartó presentar java.time como un fallo real: el proyecto usa Calendar y DateFormat.
SDK y dependencias	Se contrastaron versiones y manifiestos. Se corrigió la explicación de targetSdk y la dependencia de Play Services.
Memoria de fotografías	Se preparó el muestreo por tamaño y una prueba con el decodificador de Android. El resultado quedó en archivos verificables.
Redacción y pruebas	Se integraron los aportes corregidos y se marcaron los pendientes. No se aceptó presentar pruebas sin evidencia ni reflexiones individuales inventadas.

Lo que nos falta antes de entregar

- Completar en ambos Android los tres recorridos funcionales, con cuentas ficticias y confirmación del servidor. Preparar Firebase local como indican los protocolos del proyecto.
- Revisar con el equipo el contraste de la navegación en Android 7 y decidir si se aplica el ajuste propuesto. Revisar visualmente miniaturas, detalle y accesibilidad.
- Completar la observación de recursos del mapa y directorio. Las hipótesis de QR y certificados siguen pendientes de prueba.
- Añadir las cuatro reflexiones individuales de 100 a 150 palabras. Cada integrante debe escribir la suya a partir de su trabajo y de las pruebas que haya revisado o realizado.

No se hicieron despliegues, cambios de reglas, creación de roles remotos ni integración del taller Offline First con RTDB. Ese último requerimiento pertenece a otro trabajo y sigue pendiente.

Fuentes consultadas y archivos de apoyo

[1] Android Developers · SDK mínimo y objetivo

[2] Android Developers · WindowInsetsControllerCompat

[3] Android Developers · Carga eficiente de bitmaps

[4] Google Developers · Google Code Scanner

[5] Firebase · Dependencias de Google Play Services

[6] Android Developers · Cambios de Android 15 para target 35

[7] Repositorio oficial de osmdroid

Archivos del proyecto: README.md, ROLES.md, archivos Gradle, MainActivity.kt, estilos, informe de fusión del manifest, android-compatibility.md y evidencias-compatibilidad. También se revisaron el enunciado del taller y los aportes de Juan Pablo y Daniel. Las fuentes en línea se consultaron el 20 de octubre de 2026.